

《爬虫精进第0关知识学习笔记》

《爬虫精进第1关知识学习笔记》

《爬虫精进第2关知识学习笔记》

《爬虫精进第3关知识学习笔记》

《爬虫精进第4关知识学习笔记》

《爬虫精进第5关知识学习笔记》

《爬虫精进第6关知识学习笔记》

《爬虫精进第7关知识学习笔记》

《爬虫精进第8关知识学习笔记》

《爬虫精进第9关知识学习笔记》

《爬虫精进第10关知识学习笔记》

《爬虫精进第11关知识学习笔记》

《爬虫精进第13关知识学习笔记》

《爬虫精进第14关知识学习笔记》

《爬虫精进第15关知识学习笔记》

注:

人人都是蜘蛛侠的网站原来的链接失效，会出现不安全的链接

<https://wordpress-edu-3autumn.localprod.forc.work/wp-login.php>

现在更新成

<https://wordpress-edu-3autumn.localprod.oc.forchange.cn/wp-login.php>

其他涉及到的链接都需要改一下

账号: spiderman, 密码: crawler334566

《爬虫精进第0关知识学习笔记》

爬虫，从本质上来说，就是利用程序在网上拿到对我们有价值的¹数据。

知识一：浏览器的工作原理



知识二：爬虫的工作原理



#爬虫的四个步骤

第0步：获取数据。爬虫程序会根据我们提供的网址，向服务器发起请求，然后返回数据。

第1步：解析数据。爬虫程序会把服务器返回的数据解析成我们能读懂的格式。

第2步：提取数据。爬虫程序再从中提取出我们需要的数据。

第3步：储存数据。爬虫程序把这些有用的数据保存起来，便于你日后的使用和分析。

知识三：requests.get()

```
1 import requests #引入requests库
2 res = requests.get('URL') #requests.get是在调用requests库中的get()方法，它向服务器发送了一个请求，括号里的参数是你需要的数据所在的网址，然后服务器对请求作出了响应。
```

requests 库可以帮我们下载网页源代码、文本、图片，甚至是音频。其实，“下载”本质上是向服务器发送请求并得到响应。

```
1 import requests
2 res = requests.get('https://res.pandateacher.com/2018-12-18-10-43-07.png')
3 print(type(res))
4 #打印变量res的数据类型
```

运行结果>>> <class 'requests.models.Response'>

这代表着：res是一个对象，属于requests.models.Response类

知识四：Response对象的常用属性

Response对象的常用属性	
属性	作用
response.status_code	检查请求是否成功
response.content	把response对象转换为二进制数据
response.text	把response对象转换为字符串数据
response.encoding	定义response对象的编码

by 风变编程

#response.status_code (检查是否请求成功)

```
1 import requests
2 res = requests.get(URL)
3 print(res.status_code) #打印变量res的响应状态码，以检查请求是否成功
```

运行结果>>>200

常见响应状态码解释

响应状态码	说明	举例	说明
1xx	请求收到	100	继续提出请求
2xx	请求成功	200	成功
3xx	重定向	305	应使用代理访问
4xx	客户端错误	403	禁止访问
5xx	服务器端错误	503	服务不可用

by 风变编程

#response.content（能把Response对象的内容以二进制数据的形式返回，适用于图片、音频、视频的下载）

```
1 import requests
2 res = requests.get('https://res.pandateacher.com/2018-12-18-10-43-07.png')
3 pic=res.content #把Reponse对象的内容以二进制数据的形式返回
4
5 photo = open('ppt.jpg','wb') #新建了一个文件ppt.jpg，这里的文件没加路径，它会被保存在程序运行的当前目录下。
6 photo.write(pic) #获取pic的二进制内容
7 photo.close() #关闭文件
```

#response.text（把Response对象的内容以字符串的形式返回，适用于文字、网页源代码的下载）

```
1 import requests
2 res = requests.get('https://localprod.pandateacher.com/python-manuscript/crawler-html/sanguo.md')
3
4 novel=res.text #把Response对象的内容以字符串的形式返回
5 print(novel[:800]) #只输出800字。
```

#response.encoding （它能帮我们定义Response对象的编码）

```
1 import requests
2 res = requests.get('https://localprod.pandateacher.com/python-manuscript/crawler-html/sanguo.md')
3
4 res.encoding='gbk' #定义Response对象的编码为gbk
5 novel=res.text #把Response对象的内容以字符串的形式返回
6 print(novel[:800]) #打印小说的前800个字
```

1.什么时候用res.encoding?

目标数据本身是什么编码是未知的。用requests.get()发送请求后，我们会取得一个Response对象，其中，requests库会对数据的编码类型做出自己的判断。但是有可能不准确。

如果它判断准确的话，我们打印出来的response.text的内容就是正常的、没有乱码的，那就用不到res.encoding；如果判断不准确，就会出现一堆乱码，那我们就可以去查看目标数据的编码，然后再用res.encoding把编码定义成和目标数据一致的类型即可。

总的来说，就是遇上文本的乱码问题，才考虑用res.encoding。

知识五：查看网站的robots协议

在网站的域名后加上/robots.txt就可以了

如： <http://www.taobao.com/robots.txt>

```
1 User-agent: Baiduspider #百度爬虫
2 Allow: /article #允许访问 /article.htm
3 Allow: /oshtml #允许访问 /oshtml.htm
4 Allow: /ershou #允许访问 /ershou.htm
5 Allow: /$ #允许访问根目录，即淘宝主页
6 Disallow: /product/ #禁止访问/product/
7 Disallow: / #禁止访问除 Allow 规定页面之外的其他所有页面
8
9 User-Agent: Googlebot #谷歌爬虫
10 Allow: /article
11 Allow: /oshtml
12 Allow: /product #允许访问/product/
13 Allow: /spu
14 Allow: /dianpu
```

```
15 Allow: /oversea
16 Allow: /list
17 Allow: /ershou
18 Allow: /$
19 Disallow: / #禁止访问除 Allow 规定页面之外的其他所有页面
20
21 ..... # 文件太长，省略了对其它爬虫的规定，想看全文的话，点击上面的链接
22
23 User-Agent: * #其他爬虫
24 Disallow: / #禁止访问所有页面
```

《爬虫精进第1关知识学习笔记》

HTML (Hyper Text Markup Language) 是用来描述网页的一种语言，也叫超文本标记语言

查看网页源代码：在网页任意地方点击鼠标右键，然后点击“显示网页源代码”。
(Windows系统的电脑还可以使用快捷键ctrl+u来查看网页源代码)

查看开发者工具栏：在网页的空白处点击右键，然后选择“检查”（快捷方式是ctrl+shift+i, 或者f12)

知识一：html的组成

1.架构

```
1 <html>
2   <head>
3     <meta charset="utf-8"> #定义了HTML文档的字符编码。
4     <title>我是网页的名字</title>
5   </head>
6   <body>
7     <h1>我是一级标题</h1>
```

```
8 <h2>我是二级标题</h2>
9 <h3>我是三级标题</h3>
10 <p>我是一个段落啦。一级标题、二级标题和我，我们三个一起组成了body。
11 </p>
12 </body>
13 </html>
```

2. 标签通常是成对出现的

3. 开始标签+结束标签+中间的所有内容，它们在一起就组成了【元素】

常见HTML元素			
开始标签	元素内容	结束标签	用法
<h1>	一级标题	</h1>	一级标题
<h2>	二级标题	</h2>	二级标题
<p>	段落文本	</p>	段落
<a>	描述链接的文本		超链接
<div>	其它元素或文本	</div>	块

by 风变编程

知识二：html的属性

1. 注意：HTML的属性和Python中的属性不是一个东西，不要搞混。

#style属性（规定元素的行内样式）

可以用来定义网页文本的样式，比如字体大小、颜色、间距、对齐方式等等。

```
1 <h1 style="color:#20b2aa;">这个书苑不太冷</h1>
```

#href属性（用来定义链接）

在HTML中，链接一般都由<a>标签定义，href属性用于规定指向页面的URL

```
1 <a href="https://wordpress-edu-3autumn.localprod.forc.work/">我是一个链接，点我试试</a>
```


#class属性（为Html元素定义一个或多个类名）

```
1 <html>
2  /*网页头部分*/
3  <head>
4    <meta charset="utf-8"> #定义了HTML文档的字符编码
5    <title>这个书苑不太冷3.0</title> #定义网页的标题
6    <style>
7      .book {
8        float: left; /*控制元素浮动*/
9        margin: 5px; /*外边距为5像素*/
10       padding: 15px; /*内边距为15像素*/
11       width: 350px; /*宽度为350像素*/
12       height: 240px; /*高度为240像素*/
13       border: 3px solid #20b2aa; /*边框为3像素*/
14     }
15   </style>
16 </head>
17
18
19  /*网页体部分*/
20  <body>
21    <h1 style="color:#20b2aa;">这个书苑不太冷</h1>
22    <h3>吴枫喜欢的书:</h3>
23    <a href="https://spidermen.cn">点这里看看</a>
24    <div class="book">
25      <h2>《奇点遗民》</h2>
26      <p>本书精选收录了刘宇昆的科幻佳作共22篇。《奇点遗民》融入了科幻艺术吸引人的几大元素：数字化生命、影像化记忆、人工智能、外星访客.....刘宇昆的独特之处在于，他写的不是科幻探险或英雄奇幻，而是数据时代里每个人的生活和情感变化。透过这本书，我们看到的不仅是未来还有当下。
27    </p>
28  </div>
29 </body>
30 </html>
```

1. <style>元素的内容，是对.book的具体描述
2. .book代表class book
3. 在网页头里面，定义了class属性，属性值为"book"，然后下面一长串代码是对这个class属性的描述；接着再在网页体中调用，所以看到了<div class="book">

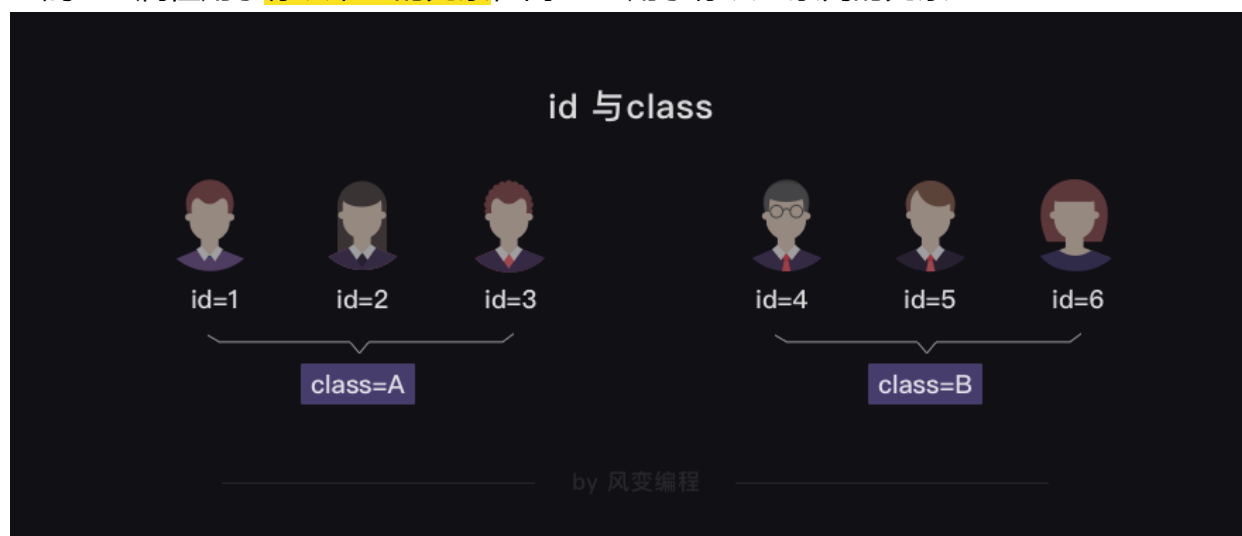
4. 在HTML中，class属性也可以被多次利用：

<https://localprod.pandateacher.com/python-manuscript/crawler-html/spider-men4.0.html>

#id属性 （定义元素的唯一id）

相同：给元素定义id和class的目的都是为了查找、定位元素，或者为元素设置样式

区别：id属性用于标识唯一的元素，而class用于标识一系列的元素



知识二：爬取html代码实例

```
1 import requests #调用requests库
2 res = requests.get('https://localprod.pandateacher.com/python-manuscript/crawler-html/spider-men5.0.html')
3 #获取网页源代码，得到的res是response对象
4
5 print(res.status_code) #检查请求是否正确响应
6
7 html = res.text #把res的内容以字符串的形式返回
8 print(html)#打印html
```

《爬虫精进第2关知识学习笔记》

知识一：BeautifulSoup是什么

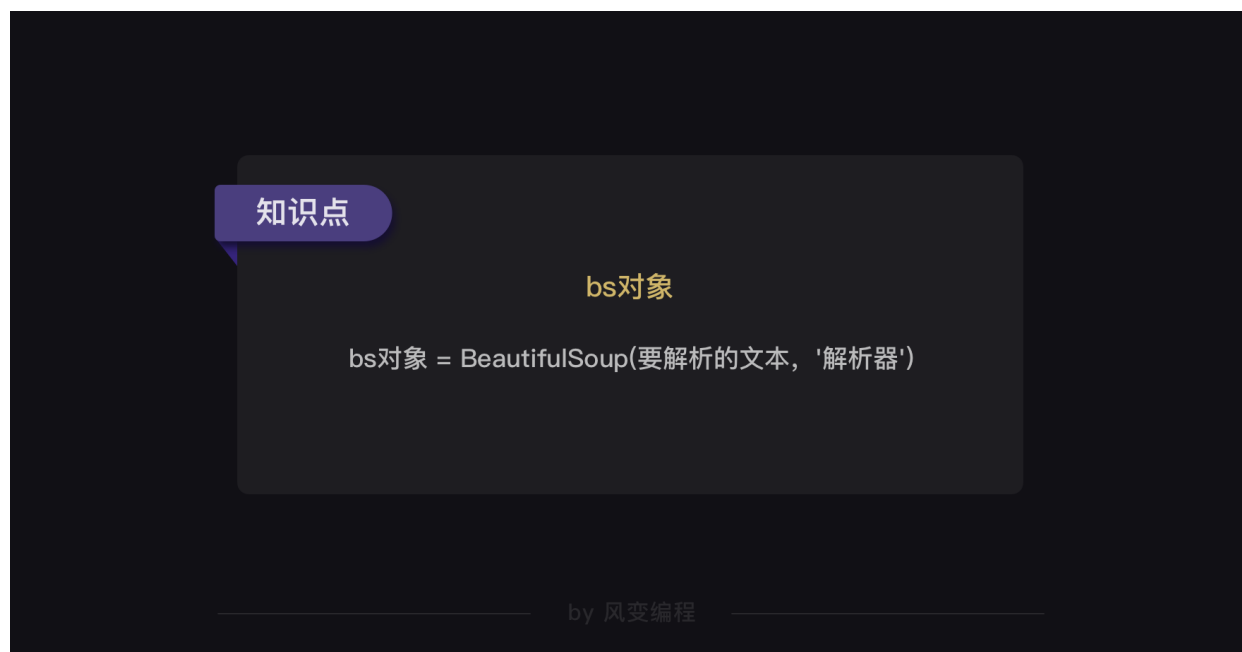
1.作用：解析和提取网页中的数据。

解析数据：把服务器返回来的HTML源代码翻译为能看懂的样子

提取数据：把我们需要的数据从众多数据中挑选出来

2.安装模块：`pip install BeautifulSoup4` (Mac电脑需要输入`pip3 install BeautifulSoup4`)

知识二：解析数据



1. 格式

第0个参数是要被解析的文本，必须是字符串

第1个参数用来标识解析器，我们要用的是一个Python内置库：html.parser（它不是唯一的解析器，但是比较简单的）

```
1 from bs4 import BeautifulSoup
2 soup = BeautifulSoup(字符串, 'html.parser')
```

实例：

```
1 import requests
2 from bs4 import BeautifulSoup #引入BS库
3
4 res = requests.get('https://localprod.pandateacher.com/python-manuscript/crawler-html/spider-men5.0.html')
```

```
5 html = res.text
6 soup = BeautifulSoup(html, 'html.parser') #把网页解析为BeautifulSoup对象
```

知识三：提取数据

#find() 和 find_all()

1. 作用

可以匹配html的标签和属性，把BeautifulSoup对象里符合要求的数据都提取出来

2.区别

find()只提取首个满足要求的数据，而find_all()提取出的是所有满足要求的数据

find() 与 find_all()的用法

方法	作用	用法	示例
find()	提取满足要求的首个数据	BeautifulSoup对象. find (标签, 属性)	soup.find ('div',class_='books')
find_all()	提取满足要求的所有数据	BeautifulSoup对象. find_all (标签, 属性)	soup.find_all ('div',class_='books')

by 风变编程

#find() 提取满足要求的首个数据

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = 'https://localprod.pandateacher.com/python-manuscript/crawler-html/spder-men0.0.html'
5 res = requests.get (url)
6 print(res.status_code)
7
8 soup = BeautifulSoup(res.text, 'html.parser')
9 item = soup.find('div') #使用find()方法提取首个<div>元素，并放到变量item里。
10
11 print(type(item)) #打印item的数据类型
12 print(item) #打印item
```

运行结果>>>

200

<class 'bs4.element.Tag'>

<div>大家好，我是一个块</div>

#find_all() 提取出的是所有满足要求的数据

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 url = 'https://localprod.pandateacher.com/python-manuscript/crawler-html/spder-men0.0.html'
5 res = requests.get(url)
6 print(res.status_code)
7
8 soup = BeautifulSoup(res.text, 'html.parser')
9 items = soup.find_all('div') #用find_all()把所有符合要求的数据提取出来，并放在变量items里
10
11 print(type(items)) #打印items的数据类型
12 print(items) #打印items
```

运行结果>>>

200

<class 'bs4.element.ResultSet'>

[<div>大家好，我是一个块</div>, <div>我也是一个块</div>, <div>我还是一个块</div>]

3.注意

find() 与 find_all() 的用法

方法	作用	用法	示例
find()	提取满足要求的首个数据	BeautifulSoup对象. find (标签, 属性)	soup.find ('div', class_='books')
find_all()	提取满足要求的所有数据	BeautifulSoup对象. find_all (标签, 属性)	soup.find_all ('div', class_='books')

by 风变编程

- 1) 举例中括号里的class_，这里有一个下划线，是为了和python语法中的类 class区分，避免程序冲突
- 2) 括号中的参数：标签和属性可以任选其一，也可以两个一起使用，这取决于我们要在网页中提取的内容。（如果只用其中一个参数就可以准确定位的话，就只用一个参数检索。如果需要标签和属性同时满足的情况下才能准确定位到我们想找的内容，那就两个参数一起使用。）

#find_all() 通过定位标签和属性提取我们想要的数

```
1 import requests # 调用requests库
2 from bs4 import BeautifulSoup # 调用BeautifulSoup库
3
4 res = requests.get('https://localprod.pandateacher.com/python-manuscript/
crawler-html/spider-men5.0.html')# 返回一个Response对象，赋值给res
5 html= res.text# 把Response对象的内容以字符串的形式返回
6
7 soup = BeautifulSoup( html, 'html.parser' ) # 把网页解析为BeautifulSoup对象
8 items = soup.find_all(class_='books') # 通过定位标签和属性提取我们想要的数
9
10 for item in items:
11     print('想找的数据都包含在这里了: \n',item) # 打印item
```

#Tag对象

Tag对象的三种常用属性与方法

属性/方法	作用
Tag.find()和Tag.find_all()	提取Tag中的Tag
Tag.text	提取Tag中的文字
Tag['属性名']	输入参数：属性名，可以提取Tag中这个属性的值

by 风变编程

```
1 items = soup.find_all(class_='books') # 通过匹配属性class='books'提取出我们想要的元素
2 for item in items: # 遍历列表items
3     kind = item.find('h2') # 在列表中的每个元素里，匹配标签<h2>提取出数据
4     title = item.find(class_='title') # 在列表中的每个元素里，匹配属性class_='title'提取出数据
5     brief = item.find(class_='info') # 在列表中的每个元素里，匹配属性class_='info'提取出数据
6     print(kind.text, '\n', title.text, '\n', title['href'], '\n', brief.text) # 打印书籍的类型、名字、链接和简介的文字
```

知识四：对象的变化过程

基础版

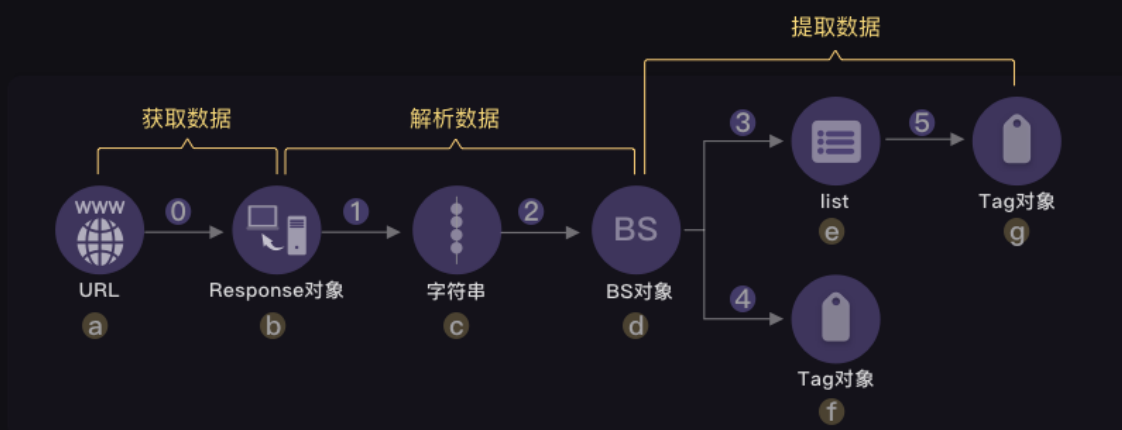


注释:

- ① request.get(url)
- ② response.text
- ③ BeautifulSoup(字符串,'html,parser')
- ④ find_all()
- ⑤ find()
- ⑥ 遍历提取

by 风变编程

升级版



注释:

- ① request.get(url)
- ② response.text
- ③ BeautifulSoup(字符串,'html,parser')
- ④ find_all()
- ⑤ find()
- ⑥ 遍历提取

- a 网址
- b Response对象
 - response.status_code
 - response.content
 - response.text
 - response.encoding
- c 字符串
- d BS对象
 - find()
 - find_all()
- e 由Tag组成的列表
- f Tag对象
 - Tag.find()和Tag.find_all()
 - Tag.text
 - Tag['属性名']
- g 同上

by 风变编程

补充知识:

1.select()

2.在bs的官方文档中，find()与find_all()的方法，其实不止标签和属性两种，还有这些：

```
find (tag, attributes, recursive, text, keywords)
find_all (tag, attributes, recursive, text, limit, keywords)
```

by 风变编程

《爬虫精进第3关知识学习笔记》

项目实操：

1. 确认目标-分析过程-代码实现，是我们做每一个项目的必经之路。
2. 将想要的数据分别提取，再做组合是一种不错的思路。但是，如果数据的数量对不上，就会让事情比较棘手。
3. 寻找最小共同父级标签是一种很常见的提取数据思路，它能够有效规避这个问题。但有时候，可能需要你反复操作，提取数据。
4. 在实际项目实操中，需要根据情况，灵活选择，灵活组合

5. text获取到的是该标签内的纯文本信息，即便是在它的子标签内，也能拿得到。但提取属性的值，只能提取该标签本身的：

```
1 from bs4 import BeautifulSoup
2
3 bs = BeautifulSoup('<p><a>惟有痴情难学佛</a>独无媚骨不如人</p>', 'html.parser')
4 tag = bs.find('p')
5 print(tag.text)
```

6. 爬虫实践当中，其实常常会因为标签选取不当，一般有两种处理方案：数量太多而无规律，我们会换个标签提取；数量不多而有规律，我们会对提取的结果进行筛选——只要列表中的若干个元素就好。

《爬虫精进第4关知识学习笔记》

知识一：Network

#什么是Network

Network的功能是：记录在当前页面上发生的所有请求。

页面的第0个请求：search.html，就是用requests.get()获取到的网页源代码

#Network怎么用

勾选框Preserve log，它的作用是“保留请求日志”。如果不点击这个，当发生页面跳转的时候，记录就会被清空。所以，我们在爬取一些会发生跳转的网页时，会点亮它。

ALL	查看全部
XHR	一种不借助刷新网页即可传输数据的对象
Doc	Document, 第0个请求一般在这里
Img	仅查看图片
Media	仅查看媒体文件
Other	其他
JS和CSS	前端代码, 负责发起请求和页面实现
Font	字体
WS和Manifest	网络编程相关知识, 无需了解

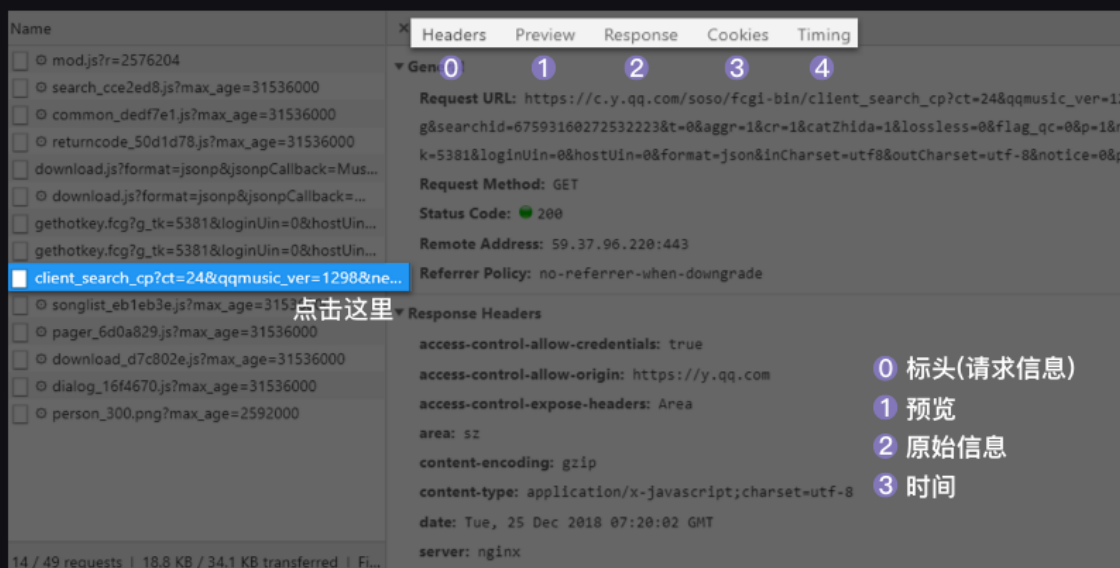
by 风变编程

知识二：XHR

#什么是XHR?

Ajax技术在工作的时候, 会创建一个XHR (或是Fetch) 对象, 然后利用XHR对象来实现, 服务器和浏览器之间传输数据

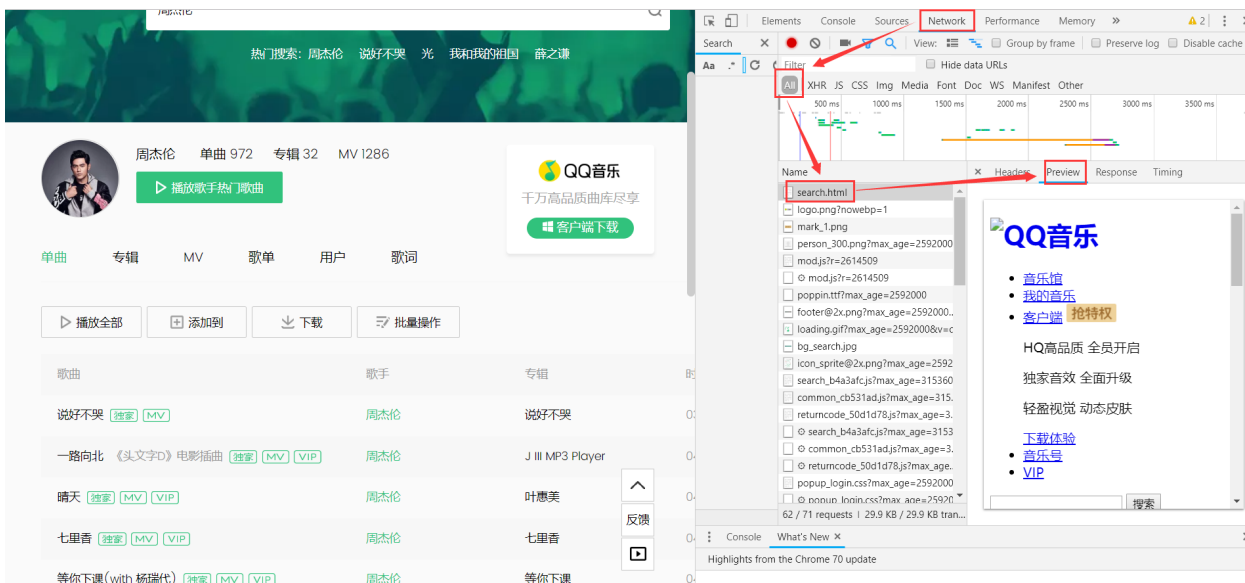
#XHR怎么请求?



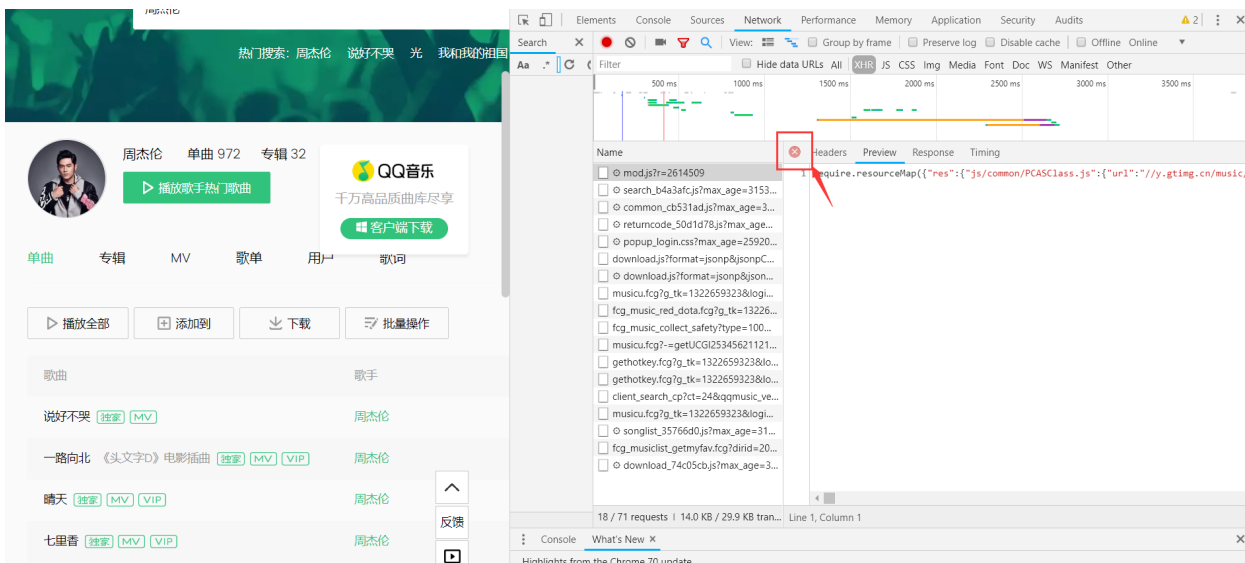
by 风变编程

从左往右分别是: Headers: 标头 (请求信息)、Preview: 预览、Response: 原始信息、Timing: 时间。

#快速查找XHR



1. 先把Network面板清空，刷新页面，查看Network多出来的新XHR；
通常，第一个请求为html请求。一般，先看一下第一个请求，如果在Preview中没有需要的数据，那么说明要爬取的内容需要通过到XHR里面寻找



2. 可以点这个X，把页面关掉；再需要点两下size，然后请求会根据size进行一个排序；
通常第一个很大可能就是我们需要数据所在的请求啦，如果第一个请求没有，再往第二个请求找

Name	Status	Type	Initiator	Size	Waterfall
client_search_cp?ct=24&qqmusic...	200	xhr	Other	10.3 KB	
gethotkey.fcgi?g_tk=1322659323...	200	xhr	Other	857 B	
gethotkey.fcgi?g_tk=1322659323...	200	xhr	Other	857 B	
musicu.fcgi?g_tk=1322659323&lo...	200	xhr	Other	514 B	
fcgi_music_collect_safety?type=10...	200	xhr	search.html	369 B	
musicu.fcgi?g_tk=1322659323&lo...	200	xhr	search.html	321 B	
fcgi_music_red_dota.fcgi?g_tk=132...	200	xhr	search.html	301 B	
musicu.fcgi?-=getUCGI253456211...	200	xhr	search.html	298 B	
fcgi_musiclist_getmyfav.fcgi?dirid=...	200	xhr	Other	277 B	
download_74c05cb.js?max_age=...	200	fetch	sw.js:1	(from disk c...	
songlist_35766d0.js?max_age=...	200	fetch	sw.js:1	(from disk c...	
download.js?format=jsonp&jso...	200	fetch	sw.js:1	(from disk c...	
download.js?format=jsonp&json...	200	xhr	common_cb531...	(from Servic...	
popup_login.css?max_age=259...	200	fetch	sw.js:1	(from disk c...	
returncode_50d1d78.js?max_ag...	200	fetch	sw.js:1	(from disk c...	
common_cb531ad.js?max_age=...	200	fetch	sw.js:1	(from disk c...	
search_b4a3afc.js?max_age=31...	200	fetch	sw.js:1	(from disk c...	
mod.js?r=2614509	200	fetch	sw.js:1	(from disk c...	

18 / 72 requests | 14.0 KB / 29.9 KB transferred | Finish: 4.9 min | DOMContentLoaded: 167 ms | Load: 410 ms

Name	Headers	Preview	Response	Cookies	Timing
client_search_cp?ct=24&qqmusic_ve...	General				
gethotkey.fcgi?g_tk=1322659323&lo...					
gethotkey.fcgi?g_tk=1322659323&lo...					
musicu.fcgi?g_tk=1322659323&logi...					
fcgi_music_collect_safety?type=100...					
musicu.fcgi?g_tk=1322659323&logi...					
fcgi_music_red_dota.fcgi?g_tk=13226...					
musicu.fcgi?-=getUCGI25345621121...					
fcgi_musiclist_getmyfav.fcgi?dirid=20...					
download_74c05cb.js?max_age=3...					
songlist_35766d0.js?max_age=31...					
download.js?format=jsonp&json...					
download.js?format=jsonp&jsonpC...					
popup_login.css?max_age=25920...					
returncode_50d1d78.js?max_age=...					
common_cb531ad.js?max_age=3...					
search_b4a3afc.js?max_age=3153...					
mod.js?r=2614509					

Request URL: https://c.y.qq.com/soso/fcgi-bin/client_search_cp?ct=24&qqmusic_ver=1298&new_json=1&remoteplace=txt.yqq.song&searchid=63439166688881893&t=0&aggr=1&cr=1&catZhida=1&lossless=0&flag_qc=0&p=1&n=10&w=%E5%91%A8%E6%9D%B0%E4%BC%A6&g_tk=1322659323&loginUin=429496084&hostUin=0&format=json&inCharset=utf8&outCharset=utf-8¬ice=0&platform=yqq.json&needNewCode=0

Request Method: GET

Status Code: 200

Remote Address: 59.37.96.220:443

Referrer Policy: no-referrer-when-downgrade

Response Headers (11)
Request Headers (11)
Query String Parameters (23)

3. 在XHR中，我们需要跟着Preview去查看是否有我们需要的内容，找到请求之后，再回到Headers；

4. Headers：被分为四个板块。找到General里的Requests URL就是我们应该去访问的链接

注意：requests.get()XHR里的URL后使用tag.text取到的，是字符串。它不是我们想要的列表/字典，数据取不出来。需要将json格式的数据转换为列表/字典

知识三：Json

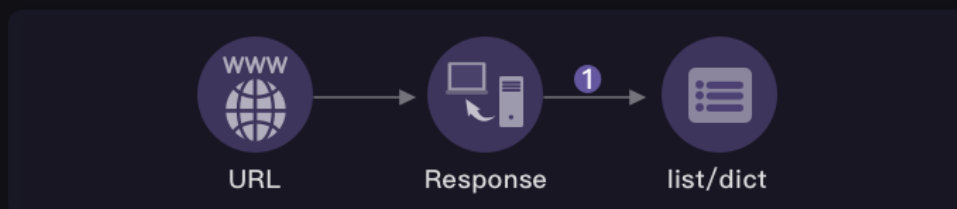
#json是什么？

json是一种特殊的字符串，这种字符串特殊在它的写法——它是用列表/字典的语法写成的

这种特殊的写法决定了，json能够有组织地存储信息。

我们总是可以将json格式的数据，转换成正常的列表/字典，也可以将列表/字典，转换成json

#json数据如何解析？



注释：

① Response类支持使用json()方法来将数据转为list/dic, 如：

```
1 import requests
2 r = requests.get('http://.....')
3 print(r.json())
```

by 风变编程

#json.loads(res.text)和直接res.json()的区别是？

没什么区别都是解析json库，只是传递的值不一样，一个是response一个需要.content, 使用 json.loads(res.text) 需要导入json模块，res.json() 是requests的方法

《爬虫精进第5关知识学习笔记》

知识一：带参数请求数据

#url的组成

前半部分大多形如：<https://xx.xx.xxx/xxx/xxx>

后半部分，多形如：xx=xx&xx=xxx&xxxxxx=xx&.....

两部分使用?来连接。

如 <https://www.zhihu.com/search?>

[type=content&q=%E5%AE%87%E5%AE%99%E5%A4%A7%E7%88%86%E7%82%B8](https://www.zhihu.com/search?type=content&q=%E5%AE%87%E5%AE%99%E5%A4%A7%E7%88%86%E7%82%B8)

前半部分是我们所请求的地址，它告诉服务器，我想访问这里。而后半部分，就是我们的请求所附带的参数，它会告诉服务器，我们想要什么样的数据。

知识二：如何带参数请求数据

#Params参数

Query String Parametres里的内容，直接复制下来，封装为一个字典，传递给params。

特别注意：要给他们打引号，让它们变字符串。

#怎么直接把给params加引号?

<https://blog.csdn.net/mouday/article/details/80460612>

```
1 a = '' 里面是你复制的query string params''
2 headers = dict([line.split(": ",1) for line in a.split("\n")])
```

#参考

```
1 params = {
2     'ct': '24',
3     'qqmusic_ver': '1298',
4     'new_json': '1',
5     'remoteplace': 'sizer.yqq.song_next',
6     'searchid': '64405487069162918',
7     't': '0',
8     'aggr': '1',
9     'cr': '1',
```

```
10 'catZhida':'1',
11 'lossless':'0',
12 'flag_qc':'0',
13 'p':1,
14 'n':'20',
15 'w':'周杰伦',
16 .....
17 }
```

知识三：什么是Request Headers?

#请求头Request Headers

服务器用于判断访问者是一个普通的用户（通过浏览器），还是一个爬虫者（通过代码）

每一个请求，都会有一个Requests Headers，我们把它称作请求头。它里面会有一些关于该请求的基本信息，比如：这个请求是从什么设备什么浏览器上发出？这个请求是从哪个页面跳转而来？

origin（中文：源头）和referer（中文：引用来源）则记录了这个请求，最初的起源是来自哪个页面。它们的区别是referer会比origin携带的信息更多些。

如果我们想告知服务器，我们不是爬虫是一个正常的浏览器，就要去修改user-agent。倘若不修改，那么这里的默认值就会是Python，会被浏览器认出来。

对于爬取某些特定信息，也要求你注明请求的来源，即origin或referer的内容

知识四：如何添加Requests Headers

点击它的官方文档，搜索“user-agent”

定制请求头

如果你想为请求添加 HTTP 头部，只要简单地传递一个 `dict` 给 `headers` 参数就可以了。

例如，在前一个示例中我们没有指定 `content-type`:

```
>>> url = 'https://api.github.com/some/endpoint'
>>> headers = {'user-agent': 'my-app/0.0.1'}

>>> r = requests.get(url, headers=headers)
```

注意: 定制 header 的优先级低于某些特定的信息源，例如:

- 如果在 `.netrc` 中设置了用户认证信息，使用 `headers=` 设置的授权就不会生效。而如果设置了 `auth=` 参数，``.netrc`` 的设置就无效了。
- 如果被重定向到别的主机，授权 header 就会被删除。
- 代理授权 header 会被 URL 中提供的代理身份覆盖掉。
- 在我们能判断内容长度的情况下，header 的 `Content-Length` 会被改写。

更进一步讲，Requests 不会基于定制 header 的具体情况改变自己的行为。只不过在最后的请求中，所有的 header 信息都会被传递进去。

注意: 所有的 header 值必须是 `string`、`bytestring` 或者 `unicode`。尽管传递 `unicode` header 也是允许的，但不建议这样做。

by 风变编程

#封装请求头

```
1 headers = {
2     'origin': 'https://y.qq.com',
3     # 请求来源，本案例中其实是不需要加这个参数的，只是为了演示
4     'referer': 'https://y.qq.com/n/yqq/song/004Z8Ihr0JIu5s.html',
5     # 请求来源，携带的信息比“origin”更丰富，本案例中其实是不需要加这个参数的，只是为了演示
6     'user-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36',
7     # 标记了请求从什么设备，什么浏览器上发出
8 }
```

#作为参数传递

```
1 res_music = requests.get(url, headers=headers, params=params)
2 # 发起请求，填入请求头和参数
```



补充知识：Python处理HTML中的转义字符

<https://blog.csdn.net/BloodyPanda/article/details/79615157>

比方说一个从网页中抓到的字符串

```
1 html = '<abc>'
```

用Python可以这样处理

```
1 import HTMLParser
2 html_parser = HTMLParser.HTMLParser()
3 txt = html_parser.unescape(html) #这样就得到了txt = '<abc>'
```

如果还想转回去，可以这样

```
1 import cgi
2 html = cgi.escape(txt) # 这样又回到了 html = '<abc>'
```

《爬虫精进第6关知识学习笔记》

知识一：存储数据的方式

两种方式：存储成csv格式文件、存储成Excel文件



知识二：csv格式文件

#csv格式文件写入



#写入_不导入csv模块:

```
1 file=open('test.csv','a+') #创建test.csv文件，以追加的读写模式
2 file.write('美国队长,钢铁侠,蜘蛛侠') #写入test.csv文件
3 file.close() #关闭文件
```

#写入_导入csv模块:

```
1 import csv #引用csv模块。
2 csv_file = open('demo.csv','w',newline='',encoding='utf-8')
3 #创建csv文件，我们要先调用open()函数，传入参数：文件名“demo.csv”、写入模式“w”、newline=''、encoding='utf-8'。
```

加newline=''参数的原因是，可以避免csv文件出现两倍的行距（就是能避免表格的行与行之间出现空白行）。
加encoding='utf-8'参数的原因是，可以避免编码问题导致的报错或乱码。

创建完csv文件后，我们要借助csv.writer()函数来建立一个writer对象

```
1 writer = csv.writer(csv_file)
2 # 用csv.writer()函数创建一个writer对象。
```

再调用writer对象的writerow()方法往csv文件里写入新的内容呢

```
1 writer.writerow(['电影','豆瓣评分'])
2 #借助writerow()函数可以在csv文件里写入一行文字 "电影"和“豆瓣评分”。
3
4 csv_file.close()
5 #写入完成后，关闭文件就大功告成啦！
```

提醒：writerow()函数里，需要放入列表参数，所以我们得把要写入的内容写成**列表**。就像['电影','豆瓣评分']。

#csv格式文件读取

csv读取的步骤

0 打开文件

调用open()函数

1 创建对象

借助reader()函数

2 读取内容

遍历reader对象

3 打印内容

print()

by 风变编程

用csv.reader()函数创建一个reader对象，通过遍历后打印

```
1 import csv
2
3 csv_file=open('demo.csv','r',newline='',encoding='utf-8')
4 reader=csv.reader(csv_file) #用csv.reader()函数创建一个reader对象
5 for row in reader:
6     print(row)
```

附：csv模块本身还有很多函数和方法，附上csv模块官方文档链接
https://yiyibooks.cn/xx/python_352/library/csv.html#module-csv

知识三：excel格式文件

#Excel格式文件写入

安装openpyxl库

window电脑：pip install openpyxl

mac电脑：pip3 install openpyxl

Excel文件写入的步骤

0 创建工作簿

利用`openpyxl.Workbook()`创建workbook对象

1 获取工作表

借助workbook对象的`active`属性

2 操作单元格

单元格：`sheet['A1']`；一行：`append()`

3 保存工作簿

`save()`

by 风变编程

通过`openpyxl.Workbook()`函数就可以创建新的工作簿

```
1 import openpyxl
2 #引用openpyxl
3
4 wb = openpyxl.Workbook()
5 #利用openpyxl.Workbook()函数创建新的workbook（工作簿）对象，就是创建新的空的Excel文件
```

创建完新的工作簿后，获取工作表

```
1 sheet = wb.active
2 #wb.active就是获取这个工作簿的活动表，通常就是第一个工作表。
3
4 sheet.title = 'new title'
5 #可以用.title给工作表重命名。现在第一个工作表的名称就会由原来默认的“sheet1”改为“new title”。
```

添加完工作表，我们就能来操作单元格，往单元格里写入内容

```
1 sheet['A1'] = '漫威宇宙'
2 #把'漫威宇宙'赋值给第一个工作表的A1单元格，就是往A1的单元格中写入了'漫威宇宙'。
```

往工作表里写入一行内容的话，就得用到`append`函数。

```
1 row = ['美国队长', '钢铁侠', '蜘蛛侠']
```

```
2 #把我们想写入的一行内容写成列表，赋值给row。
3
4 sheet.append(row)
5 #用sheet.append()就能往表格里添加这一行文字。
```

最后，保存文件

```
1 wb.save('Marvel.xlsx')
2 #保存新建的Excel文件，并命名为“Marvel.xlsx”
```

#Excel格式文件读取

Excel文件读取的步骤

0 打开工作簿

利用openpyxl.load_workbook()
创建工作簿对象

1 获取工作表

workbook对象的键，wb['sheet']

2 读取单元格

借助单元格value属性，sheet['A1'].value

3 打印单元格

print()

by 风变编程

```
1 import openpyxl
2
3 wb = openpyxl.load_workbook('Marvel.xlsx') #调用openpyxl.load_workbook()函数，打开文件。
4
5 sheet = wb['new title'] #获取“Marvel.xlsx”工作簿中名为“new title”的工作表
6
7 sheetname = wb.sheetnames
8 print(sheetname)
9 #sheetnames是用来获取工作簿所有工作表的名字的。如果你不知道工作簿到底有几个工作表，就可以把工作表的名字都打印出来
```

```
10
11 A1_cell = sheet['A1']
12 A1_value = A1_cell.value
13 print(A1_value)
14 #把“new title”工作表中A1单元格赋值给A1_cell，再利用单元格value属性，就能打印
    出A1单元格的值
15
```

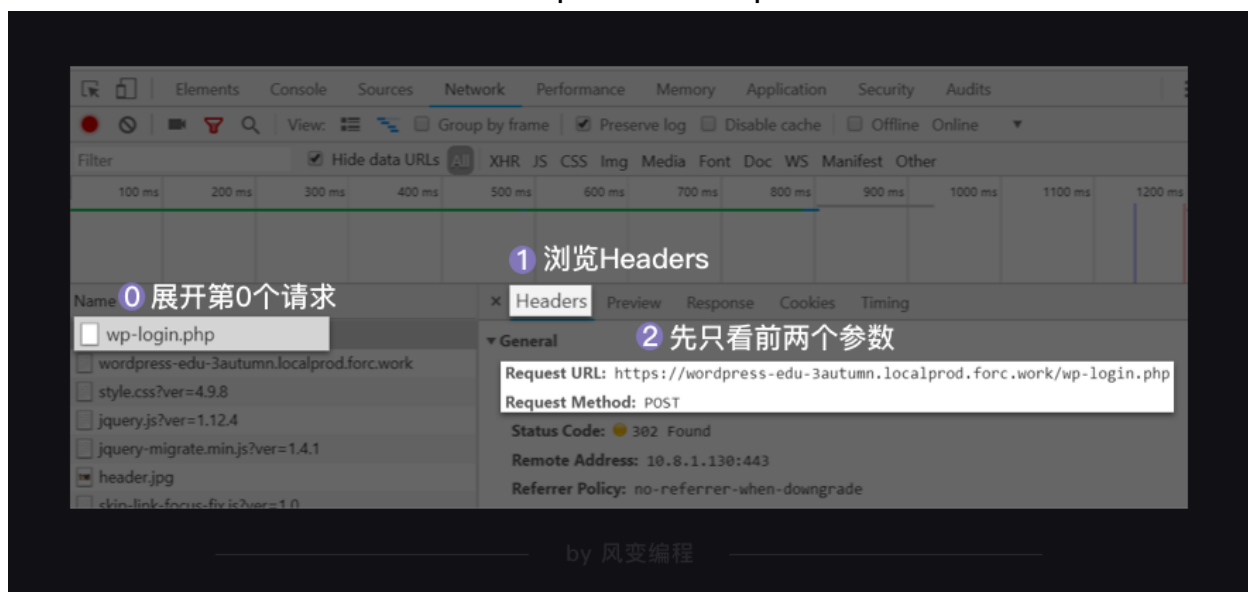
附：openpyxl模块的官方文档
<https://openpyxl.readthedocs.io/en/stable/>

《爬虫精进第8关知识学习笔记》

知识一：post请求

post请求的参数就不会直接显示，而是隐藏起来。

像账号密码这种私密的信息，就应该用post的请求，post是非明文显示



通常

get请求会应用于获取网页数据，比如我们之前学的requests.get()

post请求则应用于向网页提交数据，比如提交表单类型数据（像账号密码就是网页表单的数据）

知识二：cookies及其用法

#案例

```
1 import requests
2 #引入requests。
3
4 url = ' https://wordpress-edu-3autumn.localprod.oc.forchange.cn/wp-login.
php'
5 #把请求登录的网址赋值给url。
6
7 headers = {
8 'User-Agent':'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.3
6 (KHTML, like Gecko) Chrome/70.0.3538.110 Safari/537.36'
9 }
10 #加请求头，前面有说过加请求头是为了模拟浏览器正常的访问，避免被反爬虫。
11
12 data = {
13 'log': 'spiderman', #写入账户
14 'pwd': 'crawler334566', #写入密码
15 'wp-submit': '登录',
16 'redirect_to': 'https://wordpress-edu-3autumn.localprod.oc.forchange.cn/
wp-admin/',
17 'testcookie': '1'
18 }
19 #把有关登录的参数封装成字典，赋值给data。
20
21 login_in = requests.post(url,headers=headers,data=data)
22 #用requests.post发起请求，放入参数：请求登录的网址、请求头和登录参数，然后赋值
给login_in。
23
24 cookies = login_in.cookies
25 #提取cookies的方法：调用requests对象（login_in）的cookies属性获得登录的cooki
es，并赋值给变量cookies。
26
27 url_1 = 'https://wordpress-edu-3autumn.localprod.oc.forchange.cn/wp-comm
ents-post.php'
28 #我们想要评论的文章网址。
29
```

```

30 data_1 = {
31     'comment': input('请输入你想要发表的评论: '),
32     'submit': '发表评论',
33     'comment_post_ID': '13',
34     'comment_parent': '0'
35 }
36 #把有关评论的参数封装成字典。
37
38 comment = requests.post(url_1,headers=headers,data=data_1,cookies=cookies)
39 #用requests.post发起发表评论的请求，放入参数：文章网址、headers、评论参数、cookies参数，赋值给comment。
40 #调用cookies的方法就是在post请求中传入cookies=cookies的参数。
41
42 print(comment.status_code)
43 #打印出comment的状态码，若状态码等于200，则证明我们评论成功。

```

使用post请求的格式：

```

1 login_in = requests.post(url,headers=headers,data=data)

```

data的位置存在于，XHR里的Headers里的，Response Headers

Response Headers 存储的是服务器的响应信息，cookies就在其中。

提取cookies的方法：

```

1 cookies = login_in.cookies

```

调用cookies的方法就是在post请求中传入cookies=cookies的参数：

```

1 comment =
requests.post(url_1,headers=headers,data=data_1,cookies=cookies)

```

知识三：session及其用法

#session是什么？

session是会话过程中，服务器用来记录特定用户会话的信息

#session和cookies的关系:

cookies里带有session的编码信息, 服务器可以通过cookies辨别用户, 同时返回和这个用户相关的特定编码的session。

requests的官方文档信息:

高级用法

本篇文档涵盖了 Requests 的一些高级特性。

会话对象

会话对象让你能够跨请求保持某些参数。它也会在同一个 Session 实例发出的所有请求之间保持 cookie, 期间使用 `urllib3` 的 [connection pooling](#) 功能。所以如果你向同一主机发送多个请求, 底层的 TCP 连接将会被重用, 从而带来显著的性能提升。(参见 [HTTP persistent connection](#)).

会话对象具有主要的 Requests API 的所有方法。

我们来跨请求保持一些 cookie:

```
s = requests.Session()

s.get('http://httpbin.org/cookies/set/sessioncookie/123456789')
r = s.get("http://httpbin.org/cookies")

print(r.text)
# '{"cookies": {"sessioncookie": "123456789"}}'
```

by 风变编程

#案例

```
1 import requests #引用requests。
2
3 session = requests.session()
4 #用requests.session()创建session对象, 相当于创建了一个特定的会话, 帮我们自动保持了cookies。
5
6 url = 'https://wordpress-edu-3autumn.localprod.oc.forchange.cn/wp-login.php'
7 headers = {
8     'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36'
9 }
10
11 data = {
```

```

12  'log':input('请输入账号: '), #用input函数填写账号和密码, 这样代码更优雅, 而
    不是直接把账号密码填上去。
13  'pwd':input('请输入密码: '),
14  'wp-submit':'登录',
15  'redirect_to':'https://wordpress-edu-3autumn.localprod.oc.forchange.cn/
    wp-admin/',
16  'testcookie':'1'
17  }
18
19  session.post(url,headers=headers,data=data)
20  #在创建的session下用post发起登录请求, 放入参数: 请求登录的网址、请求头和登录参
    数。
21
22  url_1 = 'https://wordpress-edu-3autumn.localprod.oc.forchange.cn/wp-comm
    ents-post.php'
23  #把我们要评论的文章网址赋值给url_1。
24  data_1 = {
25  'comment': input('请输入你想要发表的评论: '),
26  'submit': '发表评论',
27  'comment_post_ID': '13',
28  'comment_parent': '0'
29  }
30  #把有关评论的参数封装成字典。
31
32  comment = session.post(url_1,headers=headers,data=data_1)
33  #在创建的session下用post发起评论请求, 放入参数: 文章网址, 请求头和评论参数, 并
    赋值给comment。
34
35  print(comment)
36  #打印comment

```

用requests.session()创建session对象, 相当于创建了一个特定的会话, 帮我们自动保持了cookies。

```

1  session = requests.session()

```

在创建的session下用post发起登录请求, 放入参数: 请求登录的网址、请求头和登录参数。

```

1  session.post(url,headers=headers,data=data)

```

知识四：存储cookies

#cookies类型

```
1 session.post(url,headers=headers,data=data)
2
3 print(type(session.cookies))
4 #打印cookies的类型,session.cookies就是登录的cookies
5
6 print(session.cookies)
7 #打印cookies
```

运行结果>>>

`<class 'requests.cookies.RequestsCookieJar'>`

说明：cookies是类RequestsCookieJar的实例对象，而不是字符串

思路：json模块能把字典转成字符串。我们可以先把cookies转成字典，然后再通过json模块转成字符串。这样，就能用open函数把cookies存储成txt文件。



by 风变编程

... cookies转化成字典的方法 ...

`requests.utils.dict_from_cookiejar (cj)`

从CookieJar返回键/值字典。

参数: **cj** - 从中提取cookie的CookieJar对象。

返回类型: 字典

... json模块的使用方法 ...

JSON 函数

使用 JSON 函数需要导入 json 库: `import json`。

函数	描述
<code>json.dumps</code>	将 Python 对象编码成 JSON 字符串
<code>json.loads</code>	将已编码的 JSON 字符串解码为 Python 对象

by 风变编程

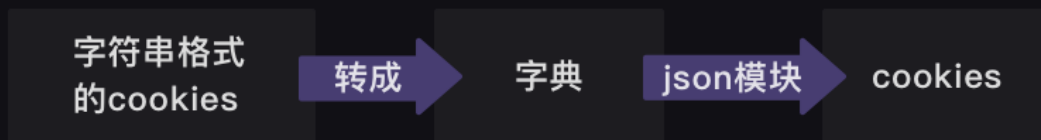
#案例

```
1 import requests,json #引入requests和json模块。
2 session = requests.session()
3
4 ... #省略
5
6 session.post(url, headers=headers, data=data)
7
8 cookies_dict = requests.utils.dict_from_cookiejar(session.cookies) #把cookies转化成字典。
9 print(cookies_dict) #打印cookies_dict
10
11 cookies_str = json.dumps(cookies_dict) #调用json模块的dumps函数,把cookies从字典再转成字符串。
12 print(cookies_str)
13
14
```

```
15 f = open('cookies.txt', 'w') #创建名为cookies.txt的文件，以写入模式写入内容。
16 f.write(cookies_str)
17 f.close()
18
```

知识五：读取cookies

思路：要先把字符串转成字典，再把字典转成cookies本来的格式



by 风变编程

#案例

```
1 cookies_txt = open('cookies.txt', 'r')
2
3 cookies_dict = json.loads(cookies_txt.read())
4 #调用json模块的loads函数，把字符串转成字典。
5
6 cookies = requests.utils.cookiejar_from_dict(cookies_dict)
7 #把转成字典的cookies再转成cookies本来的格式。
8
9 session.cookies = cookies
10 #获取cookies：就是调用requests对象（session）的cookies属性。
```

《爬虫精进第9关知识学习笔记》

知识一：selenium是什么

可以用几行代码，控制浏览器，做出自动打开、输入、点击等操作，就像是有一个真正的用户在操作一样。

可以真实地打开一个浏览器，等待所有数据都加载到Elements中之后，再把这个网页当做静态网页爬取

#安装selenium库

window电脑：pip install selenium

mac电脑：pip3 install selenium

#安装ChromeDriver驱动方法

<https://localprod.pandateacher.com/python-manuscript/crawler-html/chromedriver/ChromeDriver.html>

知识二：selenium的用法

#设置浏览器引擎

```
1 # 本地环境的浏览器设置
2 from selenium import webdriver #从selenium库中调用webdriver模块
3 driver = webdriver.Chrome() # 设置引擎为Chrome，真实地打开一个Chrome浏览器
```

#获取数据

```
1 driver.get('https://localprod.pandateacher.com/python-manuscript/hello-spiderman/') # 打开网页
2 time.sleep(1)
3 driver.close() # 关闭浏览器
```

get(URL) 是webdriver的一个方法，它的使命是为你打开指定URL的网页。

driver.close() 是关闭浏览器驱动，每次调用了webdriver之后，都要在用完它之后加上一行driver.close()用来关闭它。

#解析与提取数据

selenium库同样也具备解析数据、提取数据的能力。它和BeautifulSoup的底层原理一致，但在一些细节和语法上有所出入。

不同即是：

selenium 所解析提取的，是Elements中的所有数据，获取到的网页存在了driver中，而后，解析与提取是同时做的，都是由driver这个实例化的浏览器完成；

BeautifulSoup 所解析的则只是Network中第0个请求的响应，要把Response对象解析为BeautifulSoup对象，然后再从中提取数据

Selenium提取数据的方法

方法	作用
<code>find_element_by_tag_name</code>	通过元素的标签名称选择
<code>find_element_by_class_name</code>	通过元素的class属性选择
<code>find_element_by_id</code>	通过元素的id选择
<code>find_element_by_name</code>	通过元素的name属性选择
<code>find_element_by_link_text</code>	通过链接文本获取超链接
<code>find_element_by_partial_link_text</code>	通过链接的部分文本获取超链接

by 风变编程

#案例

```
1 # 以下方法都可以从网页中提取出'你好，蜘蛛侠！'这段文字
2
3 find_element_by_tag_name: 通过元素的名称选择
4 # 如<h1>你好，蜘蛛侠！ </h1>
5 # 可以使用find_element_by_tag_name('h1')
6
7 find_element_by_class_name: 通过元素的class属性选择
8 # 如<h1 class="title">你好，蜘蛛侠！ </h1>
9 # 可以使用find_element_by_class_name('title')
10
```

```

11 find_element_by_id: 通过元素的id选择
12 # 如<h1 id="title">你好，蜘蛛侠! </h1>
13 # 可以使用find_element_by_id('title')
14
15 find_element_by_name: 通过元素的name属性选择
16 # 如<h1 name="hello">你好，蜘蛛侠! </h1>
17 # 可以使用find_element_by_name('hello')
18
19 #以下两个方法可以提取出超链接
20
21 find_element_by_link_text: 通过链接文本获取超链接
22 # 如<a href="spidermen.html">你好，蜘蛛侠! </a>
23 # 可以使用find_element_by_link_text('你好，蜘蛛侠! ')
24
25 find_element_by_partial_link_text: 通过链接的部分文本获取超链接
26 # 如<a href="https://localprod.pandateacher.com/python-manuscript/hello-spiderman/">你好，蜘蛛侠! </a>
27 # 可以使用find_element_by_partial_link_text('你好')

```

把element换成复数elements，可以提取多个元素

Selenium提取多个数据的方法

方法	作用
find_elements_by_tag_name	通过元素的标签名称选择
find_elements_by_class_name	通过元素的class属性选择
find_elements_by_id	通过元素的id选择
find_elements_by_name	通过元素的name属性选择
find_elements_by_link_text	通过链接文本获取超链接
find_elements_by_partial_link_text	通过链接的部分文本获取超链接

by 风变编程

补充：

WebElement与Tag的用法对比

WebElement	Tag	作用
WebElement.text	Tag.text	提取文字
WebElement.get_attribute()	Tag[]	输入参数：属性名，可以提取属性值

by 风变编程

#selenium与BeautifulSoup合作

BeautifulSoup需要把字符串格式的网页源代码解析为BeautifulSoup对象，然后再从中提取数据；

selenium刚好可以获取到渲染完整的网页源代码。

使用driver的一个方法：page_source

```
1 HTML源代码字符串 = driver.page_source
```

```
1 driver.get('https://localprod.pandateacher.com/python-manuscript/hello-spiderman/') # 访问页面
2 time.sleep(2) # 等待两秒，等浏览器加缓冲载数据
3
4 pageSource = driver.page_source # 获取完整渲染的网页源代码
5 print(type(pageSource)) # 打印pageSource的类型
6 print(pageSource) # 打印pageSource
7 driver.close() # 关闭浏览器
```

运行结果>>>

<class 'str'>

使用selenium获取到的网页源代码，本身已经是字符串了

#自动操作浏览器

```
1 .send_keys() # 模拟按键输入，自动填写表单
```

```
2 .click() # 点击元素
```

```
1 driver.get('https://localprod.pandateacher.com/python-manuscript/hello-spiderman/') # 访问页面
2 time.sleep(2) # 暂停两秒，等待浏览器缓冲
3
4 teacher = driver.find_element_by_id('teacher') # 找到【请输入你喜欢的老师】下面的输入框位置
5 teacher.send_keys('必须是吴枫呀') # 输入文字
6
7 assistant = driver.find_element_by_name('assistant') # 找到【请输入你喜欢的助教】下面的输入框位置
8 assistant.send_keys('都喜欢') # 输入文字
9
10 button = driver.find_element_by_class_name('sub') # 找到【提交】按钮
11 button.click() # 点击【提交】按钮
12
13 time.sleep(1)
14 driver.close() # 关闭浏览器
```

Selenium操作元素的常用方法

方法	作用
.clear()	清除元素的内容
.send_keys()	模拟按键输入，自动填写表单
.click()	点击元素

by 风变编程

#其他

在做爬虫时，通常不需要打开浏览器，爬虫的目的是爬到数据，而不是观看浏览器的操作过程，在这种情况下，就可以使用浏览器的静默模式

```
1 # 本地Chrome浏览器的静默模式设置：
2 from selenium import webdriver #从selenium库中调用webdriver模块
3 from selenium.webdriver.chrome.options import Options # 从options模块中调用Options类
```

```
4
5 chrome_options = Options() # 实例化Option对象
6 chrome_options.add_argument('--headless') # 把Chrome浏览器设置为静默模式
7 driver = webdriver.Chrome(options = chrome_options) # 设置引擎为Chrome，在
  后台默默运行
8
```

selenium的官方文档链，目前只有英文版：

<https://seleniumhq.github.io/selenium/docs/api/py/api.html>

还可以参考这个中文文档：

<https://selenium-python-zh.readthedocs.io/en/latest/>

《爬虫精进第10关知识学习笔记》

知识一：SMTP

两个模块：email模块，smtplib模块

<https://www.cnblogs.com/lizhe860/p/9079234.html>

#smtplib模块

```
1 import smtplib # smtplib 用于邮件的发信动作
```

#email模块

email模块：支持发送的邮件内容为纯文本、HTML内容、图片、附件。email模块中有几大类来针对不同的邮件内容形式，常用如下：

MIMEText：（MIME媒体类型）内容形式为纯文本、HTML页面。

MIMEImage：内容形式为图片。

MIMEMultipart：多形式组合，可包含文本和附件。

每一类对应的导入方式：

```
1 from email.mime.text import MIMEText
2 from email.mime.image import MIMEImage
```

```
3 from email.mime.multipart import MIMEMultipart
```

#MIMEText(msg,type,charset)

msg: 文本内容, 可自定义

type: 文本类型, 默认为plain (纯文本)

charset: 文本编码, 中文为 "utf-8"

知识二：发送邮件

#案例

```
1 import smtplib
2 from email.mime.text import MIMEText
3 from email.header import Header
4 #引入smtplib、MIMETex和Header
5
6 mailhost='smtp.qq.com' #把qq邮箱的服务器地址赋值到变量mailhost上, 地址应为字符串格式
7 qqmail = smtplib.SMTP() #实例化一个smtplib模块里的SMTP类的对象, 这样就可以调用SMTP对象的方法和属性了
8 qqmail.connect(mailhost,25) #连接服务器, 第一个参数是服务器地址, 第二个参数是SMTP端口号。
9 #以上, 皆为连接服务器。
10
11 account = input('请输入你的邮箱: ') #获取邮箱账号, 为字符串格式
12 password = input('请输入你的密码: ') #获取邮箱密码, 为字符串格式
13 qqmail.login(account,password) #登录邮箱, 第一个参数为邮箱账号, 第二个参数为邮箱密码
14 #以上, 皆为登录邮箱。
15
16 receiver=input('请输入收件人的邮箱: ') #获取收件人的邮箱。
17
18 content=input('请输入邮件正文: ') #输入你的邮件正文, 为字符串格式
19 message = MIMEText(content, 'plain', 'utf-8') #实例化一个MIMEText邮件对象, 该对象需要写进三个参数, 分别是邮件正文, 文本格式和编码
20 subject = input('请输入你的邮件主题: ') #输入你的邮件主题, 为字符串格式
21 message['Subject'] = Header(subject, 'utf-8')
22 #在等号的右边是实例化了一个Header邮件头对象, 该对象需要写入两个参数, 分别是邮件主题和编码, 然后赋值给等号左边的变量message['Subject']。
23 #以上, 为填写主题和正文。
24
```

```

25 try:
26     qqmail.sendmail(account, receiver, message.as_string())
27     print ('邮件发送成功')
28 except:
29     print ('邮件发送失败')
30 qqmail.quit()
31 #以上为发送邮件和退出邮箱。
32

```

知识三：定时

#安装schedule库

window电脑: pip install schedule

mac电脑: pip3 install schedule

#案例

```

1  import schedule
2  import time
3  #引入schedule和time
4
5  def job():
6      print("I'm working...")
7      #定义一个叫job的函数，函数的功能是打印'I'm working...'
8
9  schedule.every(10).minutes.do(job) #部署每10分钟执行一次job()函数的任务
10 schedule.every().hour.do(job) #部署每x小时执行一次job()函数的任务
11 schedule.every().day.at("10:30").do(job) #部署在每天的10:30执行job()函数的任务
12 schedule.every().monday.do(job) #部署每个星期一执行job()函数的任务
13 schedule.every().wednesday.at("13:15").do(job) #部署每周三的13: 15执行函数的任务
14
15 while True:
16     schedule.run_pending()
17     time.sleep(1)
18 #15-17都是检查部署的情况，如果任务准备就绪，就开始执行任务。

```

《爬虫精进第11关知识学习笔记》

知识一：协程

原理：一个任务在执行过程中，如果遇到等待，就先去执行其他的任务，当等待结束，再回来继续之前的那个任务。在计算机的世界，这种任务来回切换得非常快速，看上去就像多个任务在被同时执行一样。

#同步

爬虫每发起一个请求，都要等服务器返回响应后，才会执行下一步。而很多时候，由于网络不稳定，加上服务器自身也需要响应的的时间，导致爬虫会浪费大量时间在等待上。这也是爬取大量数据时，爬虫的速度会比较慢的原因；

#异步

异步的爬虫方式，让多个爬虫在执行任务时保持相对独立，彼此不受干扰，可以免去等待时间，显然这样爬虫的效率和速度都会提高。

同步：



异步：

同时下载3部电影，哪一部先下载好，先看哪一部；其他电影继续保持下载状态。



by 风变编程

知识二：gevent库

用gevent库，能让我们在Python中实现多协程。

by 风变编程

1.安装gevent库

window电脑：pip install gevent

mac电脑：： pip3 install gevent

2.代码讲解

```
1 from gevent import monkey
2 #从gevent库里导入monkey模块。
3 monkey.patch_all()
4 #monkey.patch_all()能把程序变成协作式运行，就是可以帮助程序实现异步。
5 import gevent,time,requests
6 #导入gevent、time、requests。
7
8 start = time.time()
9 #记录程序开始时间。
10
11 url_list = ['https://www.baidu.com/',
12 'https://www.sina.com.cn/',
13 'http://www.sohu.com/',
14 'https://www.qq.com/',
15 'https://www.163.com/',
16 'http://www.iqiyi.com/',
17 'https://www.tmall.com/',
18 'http://www.ifeng.com/']
19 #把8个网站封装成列表。
20
21 def crawler(url):
22 #定义一个crawler()函数。
23 r = requests.get(url)
24 #用requests.get()函数爬取网站。
25 print(url,time.time()-start,r.status_code)
26 #打印网址、请求运行时间、状态码。
27
28 tasks_list = [ ]
29 #创建空的任务列表。
30
31 for url in url_list:
32 #遍历url_list。
33 task = gevent.spawn(crawler,url)
34 #用gevent.spawn()函数创建任务。
35 tasks_list.append(task)
36 #往任务列表添加任务。
37 gevent.joinall(tasks_list)
38 #执行任务列表里的所有任务，就是让爬虫开始爬取网站。
39 end = time.time()
```

```
40 #记录程序结束时间。
41 print(end-start)
42 #打印程序最终所需时间。
```

第1、3行代码：从gevent库里导入了monkey模块，这个模块能将程序转换成可异步的程序。monkey.patch_all()，它的作用其实就像你的电脑有时会弹出“是否要用补丁修补漏洞或更新”一样。它能给程序打上补丁，让程序变成是异步模式，而不是同步模式。它也叫“猴子补丁”。

我们要在导入其他库和模块前，先把monkey模块导入进来，并运行monkey.patch_all()。这样，才能先给程序打上补丁。你也可以理解成这是一个规范的写法。

第5行代码：我们导入了gevent库来帮我们实现多协程，导入了time模块来帮我们记录爬取所需时间，导入了requests模块帮我们实现爬取8个网站。

第21、23、25行代码：我们定义了一个crawler函数，只要调用这个函数，它就会执行【用requests.get()爬取网站】和【打印网址、请求运行时间、状态码】这两个任务。

第33行代码：因为gevent只能处理gevent的任务对象，不能直接调用普通函数，所以需要借助gevent.spawn()来创建任务对象。

gevent.spawn()的参数需为要调用的函数名及该函数的参数。比如，gevent.spawn(crawler,url)就是创建一个执行crawler函数的任务，参数为crawler函数名和它自身的参数url。

第35行代码：用append函数把任务添加到tasks_list的任务列表里。

第37行代码：调用gevent库里的joinall方法，能启动执行所有的任务。

gevent.joinall(tasks_list)就是执行tasks_list这个任务列表里的所有任务，开始爬取。

3.总结

多协程，是一种非抢占式的异步方式。使用多协程的话，就能让多个爬取任务用异步的方式交替执行。

用gevent实现多协程爬取的重点

- 0 定义爬取函数
- 1 用`gevent.spawn()`创建任务
- 2 用`gevent.joinall()`执行任务

by 风变编程

知识三：queue对象

1.queue是什么

queue其实是一种有序的数据结构，可以用来存取数据。

我们用多协程来爬虫，需要创建大量任务时，我们可以借助queue模块。

queue对象的方法

put_nowait()	往队列里存储数据
get_nowait()	从队列里提取数据
empty()	判断队列是否为空
full()	判断队列是否为满
qsize()	判断队列还剩多少数量

by 风变编程

2.queue模块的重点

用queue模块的重点

- 0 用Queue()创建队列
- 1 用put_nowait()存储数据
- 2 用get_nowait()提取数据

by 风变编程

3.具体的代码分析见课程，代码书写分为4个部分：

第1部分，导入模块。

第2部分，创建队列，把任务存储进队列里。

第3部分，定义爬取函数，从队列里提取出刚刚存储进去的网址

第4部分，让爬虫用多协程执行任务

《爬虫精进第13关知识学习笔记》

知识一：Scrapy是什么？

爬虫需要涉及的功能，比如导入和操作不同的模块、麻烦的异步，在Scrapy框架可以自动实现

在Scrapy里，整个爬虫程序的流程都不需要我们去操心，且Scrapy中的程序全部都是异步模式，所有的请求或返回的响应都由引擎自动分配去处理。

#Scrapy的结构

Scrapy Engine（引擎）就是最中心位置的，负责统筹公司的4大部门，每个部门都只听从它的命令，并只向它汇报工作

Scheduler（调度器）部门主要负责处理引擎发送过来的requests对象（即网页请求的相关信息集合，包括params, data, cookies, request headers...等），会把请求的url以有序的方式排列成队，并等待引擎来提取（功能上类似于gevent库的queue模块）。

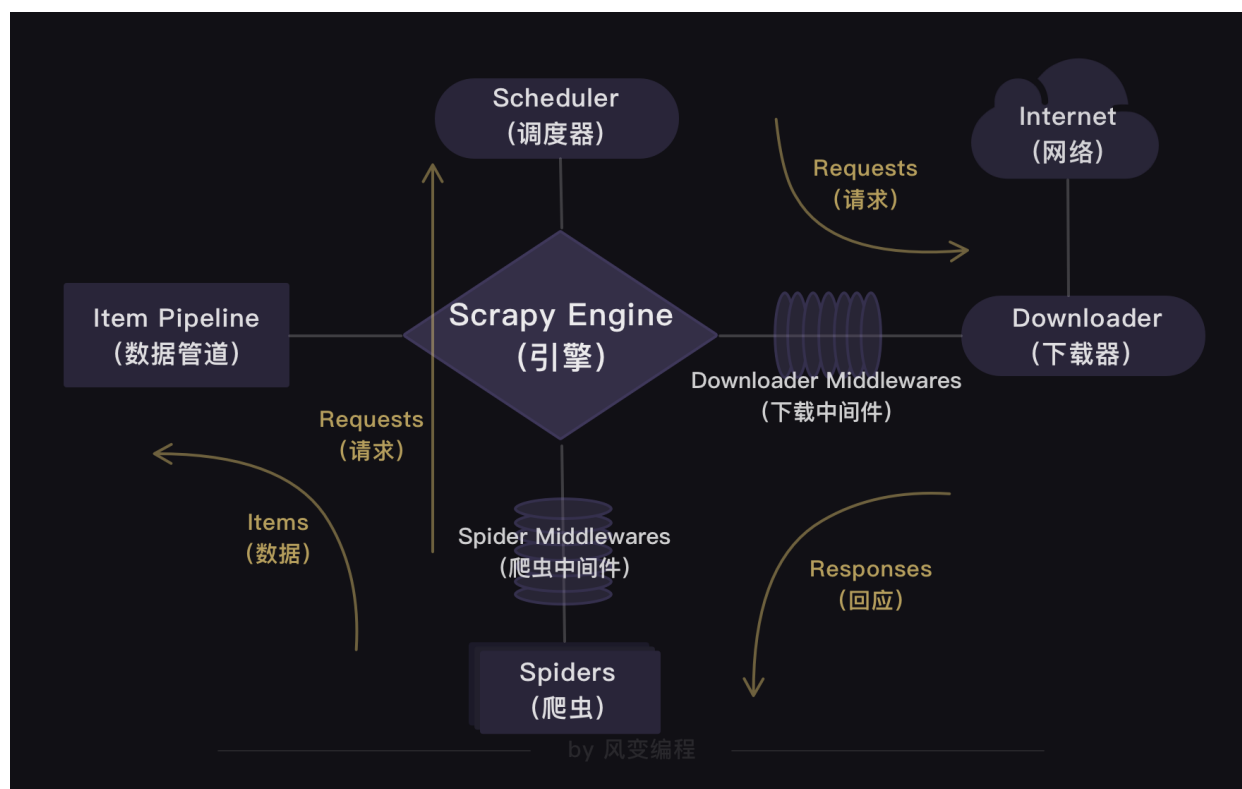
Downloader（下载器）部门则是负责处理引擎发送过来的requests，进行网页爬取，并将返回的response（爬取到的内容）交给引擎。它对应的是爬虫流程【获取数据】这一步。

Spiders（爬虫）部门是公司的核心业务部门，主要任务是创建requests对象和接受引擎发送过来的response（Downloader部门爬取到的内容），从中解析并提取出有用的数据。它对应的是爬虫流程【解析数据】和【提取数据】这两步。

Item Pipeline（数据管道）部门则是公司的数据部门，只负责存储和处理Spiders部门提取到的有用数据。这个对应的是爬虫流程【存储数据】这一步。

Downloader Middlewares（下载中间件）的工作相当于下载器部门的秘书，比如会提前对引擎大boss发送的诸多requests做出处理。

Spider Middlewares（爬虫中间件）的工作则相当于爬虫部门的秘书，比如会提前接收并处理引擎大boss发送来的response，过滤掉一些重复无用的东西。



#Scrapy的工作原理

在Scrapy里，Scrapy中的程序全部都是异步模式，所有的请求或返回的响应都由引擎自动分配去处理

哪怕有某个请求出现异常，程序也会做异常处理，跳过报错的请求，继续往下运行程序

Scrapy的用法

- 0 创建Scrapy项目
- 1 定义item(数据)
- 2 创建和编写spiders文件
- 3 修改settings.py文件
- 4 运行Scrapy爬虫

by 风变编程

知识二：Scrapy的用法

1.创建项目 —— 终端

安装scrapy库

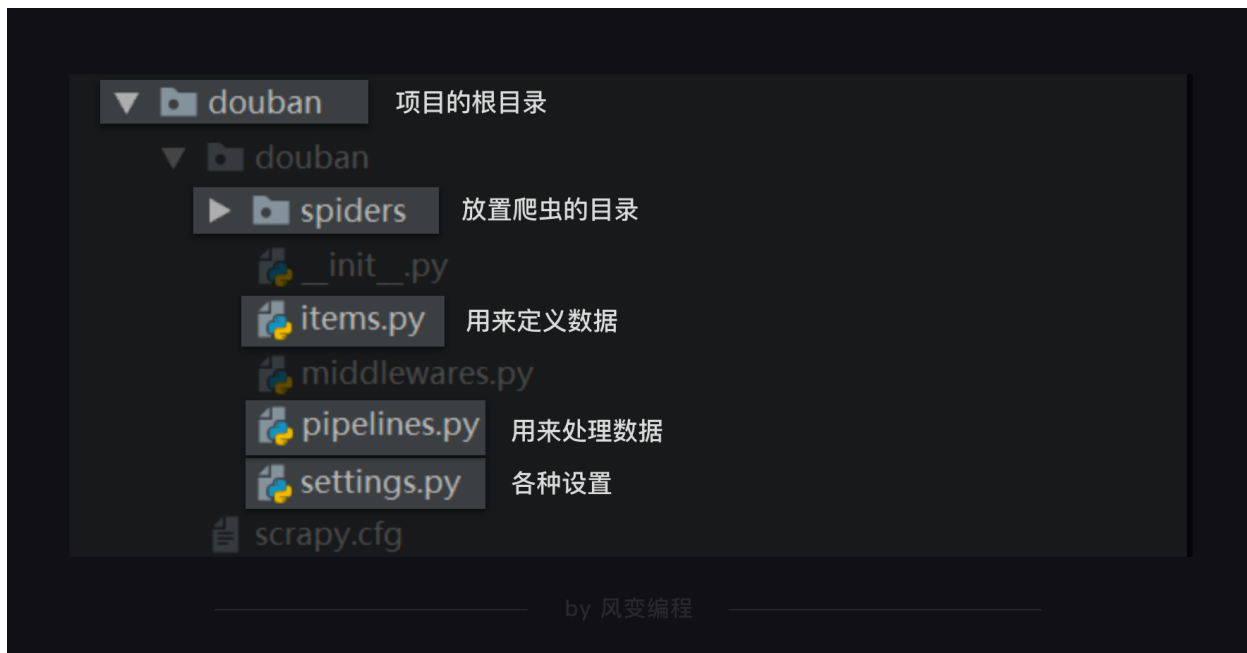
window电脑: `pip install scrapy`

mac电脑: `pip3 install scrapy`

首先，要在本地电脑打开终端，通过cd进入想要保存项目的目录下

再输入一行能帮我们创建Scrapy项目的命令：`scrapy startproject xxx (项目名)`

整个scrapy项目的结构，如下图所示



2.编辑爬虫 —— spiders文件夹下新建项目

```
1 import scrapy
2 import bs4
3 from ..items import DoubanItem
4
5 class DoubanSpider(scrapy.Spider):
6     name = 'douban'
7     allowed_domains = ['book.douban.com']
8     start_urls = ['https://book.douban.com/top250?start=0']
9
10    def parse(self, response):
11        print(response.text)
```

第3行代码：我们需要引用DoubanItem，它在items里面。因为是items在top250.py的上一级目录，所以要用..items，这是一个固定用法。

第5行代码：定义一个爬虫类DoubanSpider。DoubanSpider类继承自scrapy.Spider类。

第6行代码：name是定义爬虫的名字，这个名字是爬虫的唯一标识。name = 'douban'意思是定义爬虫的名字为douban。启动爬虫的时候，要用到这个名字。

第7行代码：allowed_domains是定义允许爬虫爬取的网址域名（不需要加https://）。如果网址的域名不在这个列表里，就会被过滤掉。

第8行代码：start_urls是定义起始网址，就是爬虫从哪个网址开始抓取。在此，allowed_domains的设定对start_urls里的网址不会有影响

第10行代码：parse是Scrapy里默认处理response的一个方法，中文是解析

3.定义数据 —— items.py

```
1 import scrapy #导入scrapy
2
3 class DoubanItem(scrapy.Item):
4     #定义一个类DoubanItem，它继承自scrapy.Item
5     title = scrapy.Field()
6     #定义书名的数据属性
7     publish = scrapy.Field()
8     #定义出版信息的数据属性
9     score = scrapy.Field()
10    #定义评分的数据属性
```

第1行代码：我们导入了scrapy，在编辑爬虫所创建的类将直接继承scrapy中的scrapy.Item类

第3行代码：我们定义了一个DoubanItem类。它继承自scrapy.Item类。

第5、7、9行代码：我们定义了书名、出版信息和评分三种数据。scrapy.Field()这行代码实现的是，让数据能以类似字典的形式记录

4.设置 —— setting.py

```
1 # Crawl responsibly by identifying yourself (and your website) on the user-agent
2 #USER_AGENT = 'douban (+http://www.yourdomain.com)'
```

```
3
4 # Obey robots.txt rules
5 ROBOTSTXT_OBEY = True
```

把 USER_AGENT 的注释取消（删除#），然后替换掉user-agent的内容，就是修改了请求头

把 ROBOTSTXT_OBEY=True 改成 ROBOTSTXT_OBEY=False，就是把遵守robots协议换成无需遵从robots协议，这样Scrapy就能不受限制地运行

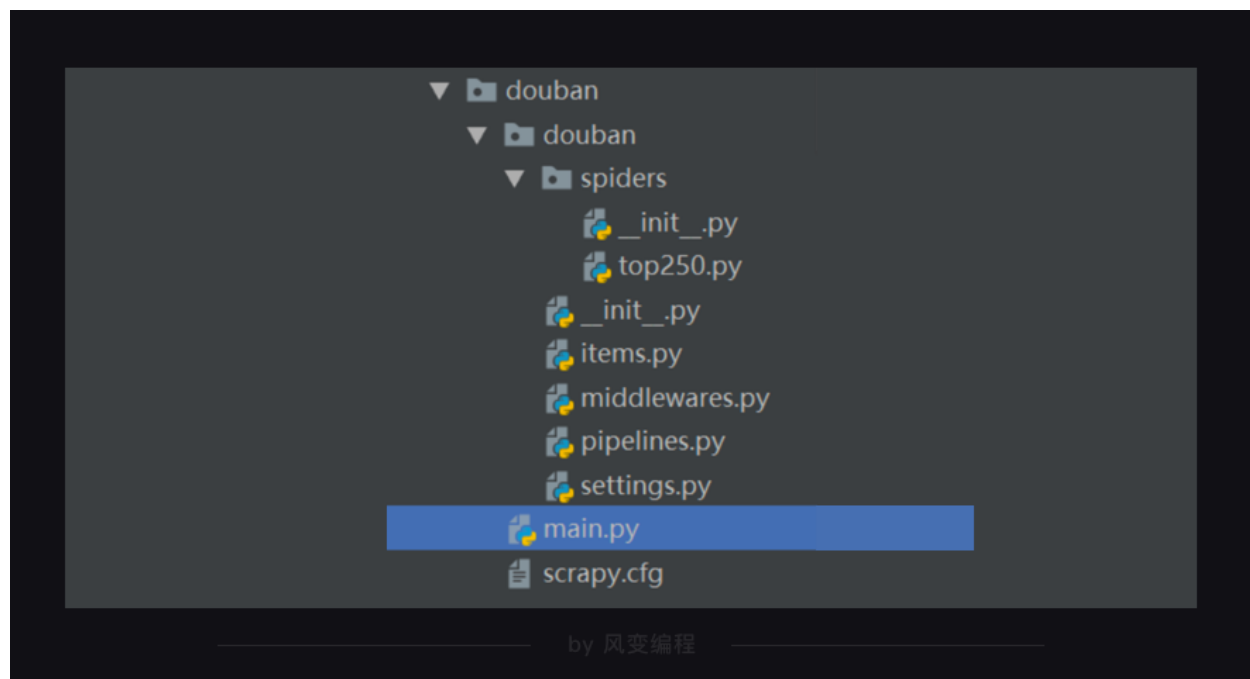
5.运行 —— 终端 / main.py

#方式1

在本地电脑的终端跳转到scrapy项目的文件夹（跳转方法：cd+文件夹的路径名），然后输入命令行：**scrapy crawl xxx（项目名）**

#方式2

在最外层的大文件夹里新建一个main.py文件（与scrapy.cfg同级）



```
1 from scrapy import cmdline
2 #导入cmdline模块,可以实现控制终端命令行。
3
4 cmdline.execute(['scrapy', 'crawl', 'douban'])
```

第1行代码：在Scrapy中有一个可以控制终端命令的模块cmdline。导入了这个模块，我们就能操控终端。

第4行代码：在cmdline模块中，有一个execute方法能执行终端的命令，不过这个方法需要传入列表的参数。我们想输入运行Scrapy的代码 `scrapy crawl douban`，就需要写成 `['scrapy','crawl','douban']` 这样

6.总结



附：Scrapy的安装问题

Unknown command:crawl 报错 在vscode中的解决办法：
https://blog.csdn.net/weixin_44520259/article/details/88569673

在Pycharm中运行Scrapy爬虫项目的基本操作：
<https://www.cnblogs.com/lssx/p/8378832.html>

Scrapy环境搭建（基于windows）：
<https://www.jianshu.com/p/ef46d878dcf7>
<https://blog.csdn.net/dvivily/article/details/81325337>

Microsoft Visual C++ 14.0 is required问题解决方案:
<https://blog.csdn.net/heyshheyoun/article/details/82022948>
<http://www.cppcns.com/jiaoben/python/205829.html>

添加环境变量:
<https://shimo.im/docs/3ph9gypjw33twJyR/read>

《爬虫精进第14关知识学习笔记》

知识一：存储文件

#存储成csv文件

方法比较简单，只需在settings.py文件里，添加如下的代码即可。

```
1 FEED_URI = './storage/data/%(name)s.csv'
2 FEED_FORMAT = 'CSV'
3 FEED_EXPORT_ENCODING = 'ansi'
```

FEED_URI是导出文件的路径。 './storage/data/%(name)s.csv'，就是把存储的文件放到与settings.py文件同级的storage文件夹的data子文件夹里。

FEED_FORMAT 是导出数据格式，写CSV就能得到CSV格式。

FEED_EXPORT_ENCODING 是导出文件编码，ansi是一种在windows上的编码格式，你也可以把它变成utf-8用在mac电脑上。

#存储成excel文件

先在setting.py里设置启用ITEM_PIPELINES，设置方法如下：

```
1 #取消`ITEM_PIPELINES`的注释后：
2
```

```

3 # Configure item pipelines
4 # See https://doc.scrapy.org/en/latest/topics/item-pipeline.html
5 ITEM_PIPELINES = {
6     'jobuittest.pipelines.JobuittestPipeline': 300,
7 }

```

编辑pipelines.py文件。存储为Excel文件，用openpyxl模块来实现，代码如下：

```

1 import openpyxl
2
3 class JobuiPipeline(object):
4     #定义一个JobuiPipeline类，负责处理item
5     def __init__(self): #初始化函数 当类实例化时这个方法会自启动
6         self.wb = openpyxl.Workbook() #创建工作簿
7         self.ws = self.wb.active #定位活动表
8         self.ws.append(['公司', '职位', '地址', '招聘信息']) #用append函数往表格添加表头
9
10    def process_item(self, item, spider): #process_item是默认的处理item的方法，就像parse是默认处理response的方法
11        line = [item['company'], item['position'], item['address'], item['detail']] #把公司名称、职位名称、工作地点和招聘要求都写成列表的形式，赋值给line
12
13        self.ws.append(line) #用append函数把公司名称、职位名称、工作地点和招聘要求的数据都添加进表格
14
15        return item #将item丢回给引擎，如果后面还有这个item需要经过的itempipeline，引擎会自己调度
16
17    def close_spider(self, spider):
18        #close_spider是当爬虫结束运行时，这个方法就会执行
19        self.wb.save('./jobui.xlsx') #保存文件
20        self.wb.close() #关闭文件

```

知识二：控制爬虫的速度

我们需要在 setting.py 里取消 DOWNLOAD_DELAY = 0 这行的注释（删掉#）。

DOWNLOAD_DELAY 翻译成中文是下载延迟的意思，这行代码可以控制爬虫的速度。因

为这个项目的爬取速度不宜过快，我们要把下载延迟的时间改成 0.5 秒。

```
1 # Configure a delay for requests for the same website (default: 0)
2 # See https://doc.scrapy.org/en/latest/topics/settings.html#download-delay
3 # See also autothrottle settings and docs
4 DOWNLOAD_DELAY = 0.5
```

《爬虫精进第15关知识学习笔记》

爬虫进阶路线指引

#解析与提取

学习正则表达式（re模块）。正则表达式功能强大，它能让你自己设定一套复杂的规则，然后把目标文本里符合条件的相关内容给找出来

#存储

从MySQL和MongoDB这两个库开始学起，它们一个是关系型数据库的典型代表，一个是非关系型数据库的典型代表

#数据分析与可视化

推荐的模块与库：Pandas / Matplotlib / Numpy / Scikit-Learn / Scipy

#更多的爬虫

分布式爬虫，需要用到框架

#更强大的爬虫——框架

使用Scrapy模拟登录、存储数据库、使用HTTP代理、分布式爬虫.....

反爬虫应对策略汇总

有的网站会限制请求头，即Request Headers，那我们就去填写 user-agent 声明自己的身份，有时还要去填写 origin 和 referer 声明请求的来源

有的网站会限制登录，不登录就不给你访问。那我们就用 cookies 和 session 的知识去模拟登录